

Multi-Agent AI Task Orchestrator

Project guide: how it works, why each piece was chosen, the alternatives, and the important syntax in the real code.

Built with	Numbers
LangGraph · LangChain · FastAPI · React (Vite) · ChromaDB · SQLite	9 agents · 29 tools · ~4,700 lines of code
LLMs: Groq gpt-oss-120b / 20b → Gemini 3.8 Flash / 3.5 Flash-Lite	Phases 0–10 + 6 follow-ups, all complete

Source: LEARNING.md and the project code · 30 September 2026

Contents

1. The project at a glance	4
What it can do	4
Around the agents	4
2. Architecture flow	5
The supervisor pattern	5
3. Life of one request	6
Step by step	6
4. Technology choices	7
5. How it was built — phase by phase	8
Phase 1 — The smallest LangGraph	8
Phase 2 — Orchestrator and routing	8
Phase 3.1 — RAG agent	9
Phase 3.2 — Tool calling (Document agent)	10
Phase 3.3 / 3.4 — Excel and PPT agents	10
Phase 3.5 — Browser agent (untrusted input)	11
Phase 3.6 / 3.7 — Email and Calendar agents	11
Phase 4 — Agent-to-agent workflows	12
Phase 5 — Tools as a system	12
Phase 6 — FastAPI backend	13
Phase 7 — React frontend (EDITH)	13
Phase 8 — Memory, state and persistence	14
Phase 9 — Human approval (human-in-the-loop)	14
Phase 10 — Reliability	15
6. Follow-up features	17
User accounts	17
File locking and atomic writes	17
Safe Markdown in the UI	17
Edit a draft before approving	18
Long documents and long conversations	18
Designed presentations	18
Uploading documents (workspace + RAG)	20
Document Editor (templates and surgical edits)	21
7. Security summary	23
8. Key lessons from the bugs we hit	24
9. Running the project	25
Keys needed (.env)	25
Commands	25

Project files	25
-------------------------	----

1. The project at a glance

You type a request in plain English. An orchestrator decides which specialist agents are needed, runs them (one after another, or several at once), asks you before anything irreversible, and returns one answer — while the UI shows every step live.

What it can do

Agent	What it does	Key limit / safety
Research	Answers general, stable-knowledge questions.	No tools, no internet.
RAG	Answers from company documents and your uploaded documents, and cites the file.	Says “Not found” instead of guessing.
Document	Reads PDF/DOCX/TXT, writes and appends .docx, map-reduce summary of long files.	Only plain file names inside workspace/.
Editor	Rewrites a PDF/document into your template or letterhead .docx; surgical find-and-replace edits.	Keeps logo, header, footer, formatting; saves a new file.
Excel	Reads sheets, computes totals per group, writes sheets, formulas, charts.	Python does the maths, never the LLM.
PPT	Designed 16:9 decks: 4 colour themes, 6 layouts (bullets, two columns, big-number stats, native charts, quote, section).	Max 6 bullets, each < 120 chars; delete needs approval.
Browser	Web search (Tavily) and reading pages, with source URLs.	Read-only; SSRF guard; page text is untrusted.
Email	Lists, searches, reads mail, writes drafts.	No send tool — you approve every send.
Calendar	Lists events, finds free slots, proposes changes.	Only proposes — you approve every change.

Around the agents

- **Workflows:** agents chain (Document → PPT) or run in parallel (RAG + Research) in the same request.
- **Human approval:** the run pauses before sending, cancelling or deleting; you can edit a draft first.
- **Memory:** follow-up questions in the same thread, a user profile (name, sign-off), history of missions, long chats compacted into a summary.
- **Accounts:** login, per-user workspace / mailbox / calendar / approvals.
- **Reliability:** four-model fallback with cooldowns, retries, per-agent crash isolation, output verification, time and loop limits, and a JSON-lines trace of every run.
- **UI:** live orchestrator ring, Mission Log, safe Markdown answers, file previews and downloads.

Not done yet (both optional)

- Interactive browsing with Playwright (needs ~150 MB Chromium) — today the Browser agent only searches and reads.
- Real Gmail / Google Calendar (needs your own Google Cloud OAuth client) — today they use sample JSON data.

2. Architecture flow

The request travels down one straight path. Everything intelligent lives in the LangGraph graph; the API and UI stay thin.

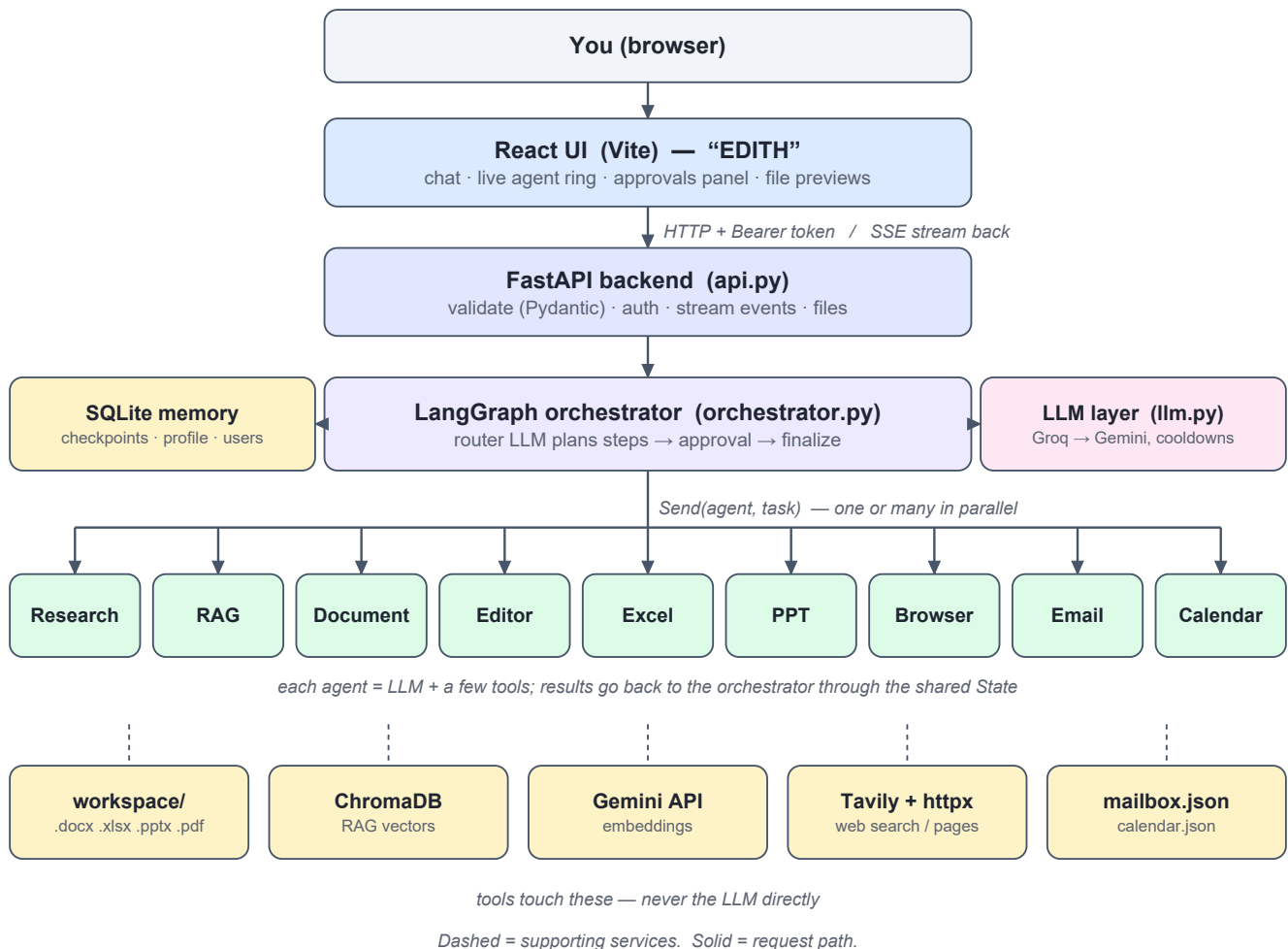


Figure 1 — System architecture: request path (solid) and supporting services (dashed).

The supervisor pattern

Agents never call each other. Each one reports back to the orchestrator, which decides the next step. All hand-offs go through the shared **State**, so every step is recorded — that is what makes pause/resume, streaming and tracing possible.

Why we used it

- One place makes every decision, so it is easy to debug and to test.
- The orchestrator can re-plan after each turn (e.g. skip the email if the calendar shows you are busy).
- Cost: one extra LLM call per turn.

Alternatives we could have used

- **Plan-then-execute**: the LLM writes the whole plan once — cheaper, but cannot react to results.
- **Agents talking to each other** (AutoGen style): flexible, but less predictable.
- **Keyword or embedding routing**: cheaper, but breaks on new wording or multi-part questions.

3. Life of one request

What happens between pressing Launch and seeing the final answer.

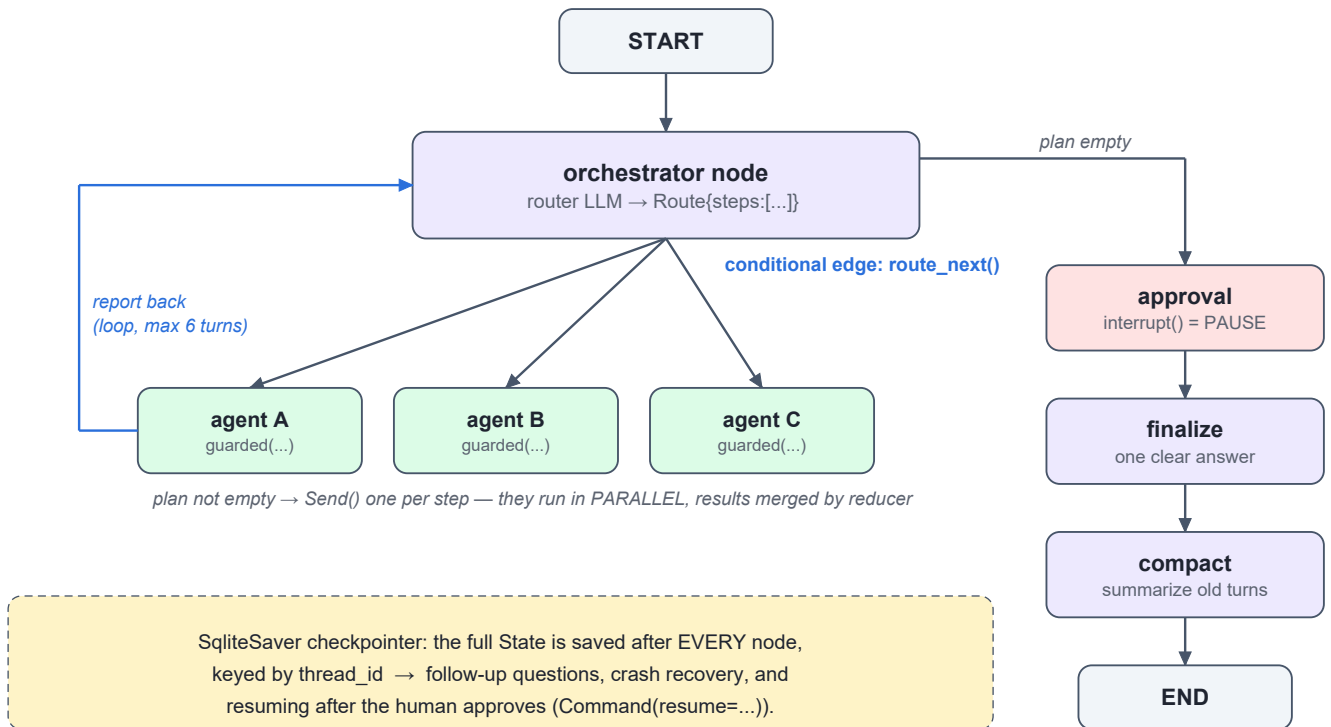


Figure 2 — The LangGraph graph inside orchestrator.py.

Step by step

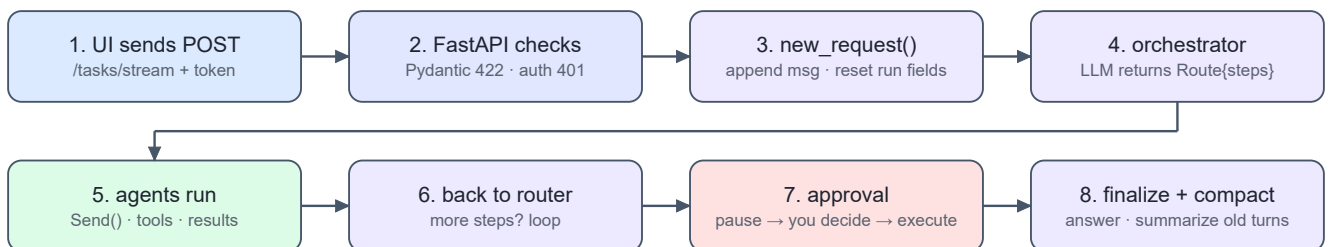


Figure 3 — Request lifecycle. Each step becomes an SSE event the UI draws live: thread → plan → agent → approval / final → files.

- **Streaming:** `graph.stream(..., stream_mode="updates")` yields `{node: output}` after every node; the API turns each into an SSE event.
- **Pause:** the approval node calls `interrupt()`. State is saved in SQLite, the stream ends with an `approval` event, and `POST /tasks/{id}/resume` continues the same run later.
- **Files:** the workspace is snapshotted before and after the run, so the UI gets the exact list of files created or changed.

4. Technology choices

Every major dependency, why it was picked, and what else would have worked.

Technology	Used for	Why we chose it	Alternatives
LangGraph	The workflow graph	Loops, branches, parallel steps, pause/resume and checkpoints built in.	Plain while-loop (build pausing yourself) · CrewAI (hides flow) · AutoGen (less predictable) · vendor SDKs (one provider)
LangChain	LLM interface, @tool, structured output	One interface for Groq and Gemini; tool schemas generated from Python functions.	Provider SDKs directly (write schemas yourself) · LiteLLM
Groq	Main LLMs (gpt-oss-120b / 20b)	Free tier, very fast, good at tool calling.	Gemini only · OpenRouter · paid Claude / OpenAI (strongest, costs money)
Gemini API	Fallback LLMs + RAG embeddings	Free tier, separate quota from Groq, strong embeddings (3,072-dim).	OpenAI / Cohere / Voyage embeddings (own key) · local MiniLM (rejected: no local models)
ChromaDB	Vector store for RAG	pip install, no server, saves to disk.	FAISS (library only) · pgvector (needs Postgres) · Pinecone (hosted, paid)
FastAPI	HTTP API + SSE streaming	Python like the graph, Pydantic validation, auto docs at /docs.	Flask (weaker async) · Django (too heavy for an API)
React + Vite	The EDITH UI	Re-renders from state — ideal for a UI that changes on every event.	Streamlit / Gradio (limited custom visuals) · plain JS (messy) · Next.js (SSR not needed)
SQLite	Checkpoints, profile, threads, users	One file, no server, survives restarts.	MemorySaver (RAM only) · PostgresSaver (multi-process production)
Tavily	Web search	Built for AI agents, clean results, 1,000 free credits / month.	ddgs / DuckDuckGo (unofficial, breaks) · Brave · SerpAPI
httpx + BeautifulSoup	Downloading and reading pages	We control redirects, timeout and size (security).	Tavily extract (costs credits) · trafilatura · Playwright (for clicking)
python-docx / pypdf	Word and PDF	Pure Python, light.	pdfplumber (tables) · OCR for scans
openpyxl	Excel	Reads and writes .xlsx, formulas and charts.	xlsxwriter (write-only) · xlwings (needs Excel) · pandas (heavy)
python-pptx	PowerPoint	Pure Python; shapes, themes and native editable charts drawn by our code.	Google Slides API (OAuth) · Aspose (paid)
react-markdown	Rendering answers	Builds React elements, never innerHTML; skipHtml drops raw HTML.	marked + DOMPurify (only as safe as the sanitiser)
hashlib.scrypt	Password hashing	Standard library, deliberately slow and memory-hard.	bcrypt / argon2 (need a package) · OAuth login

5. How it was built — phase by phase

For each phase: the problem, why this solution, the alternatives, and the important syntax from the real code.

Phase 1 — The smallest LangGraph

Goal: learn the building blocks with **START** → **agent** → **END**. A graph is overkill for one step, but Phase 2 adds branching, which the graph then handles for free.

Important syntax

```
phase1_hello_graph.py

class State(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]

def agent(state: State) -> dict:
    reply = llm.invoke(state["messages"])
    return {"messages": [reply]}          # only the key we changed

builder = StateGraph(State)
builder.add_node("agent", agent)
builder.add_edge(START, "agent")
builder.add_edge("agent", END)
graph = builder.compile()
result = graph.invoke({"messages": [HumanMessage("Say hello")]})
```

- › **TypedDict** — declares the shape of the State — the data that flows between nodes.
- › **Annotated[list, add_messages]** — attaches a **reducer**: new messages are appended instead of replacing the list.
- › **return {"messages": [...]}** — a node returns only what it changed; LangGraph merges it using the reducer.
- › **add_edge / START / END** — fixed connections and the built-in entry and exit points.
- › **compile()** — validates the graph (no dead ends) and makes it runnable; **invoke()** runs START→END.

Phase 2 — Orchestrator and routing

The LLM reads the request and the results so far, then picks the next agent(s). Structured output makes it impossible to name an agent that doesn't exist.

Why we used it

- An LLM router understands any wording and multi-part questions.
- Adding an agent is one line in the AGENTS registry — Route, prompt and graph are generated from it.

Important syntax

```
orchestrator.py
```

```

AGENTS = {
    "rag_agent": (rag_agent, "search the company knowledge base ..."),
    "excel_agent": (excel_agent, "spreadsheets (.xlsx): read data, calculate ..."),
    ...
}

class Step(BaseModel):
    agent: Literal[*AGENTS] # only real agent names are allowed
    task: str = Field(description="Clear, self-contained instruction.")
    inputs: list[Literal[*AGENTS]] = Field(default_factory=list)

class Route(BaseModel):
    steps: list[Step] # empty list = request is complete
    reason: str

router = get_llm(schema=Route) # -> .with_structured_output(Route)

```

- › `Literal[*AGENTS]` — unpacks the dict keys into allowed values; the JSON schema sent to the LLM lists only those names.
- › `Field(description=...)` — the description becomes part of the schema — effectively part of the prompt.
- › `default_factory=list` — a fresh empty list per object (never share a mutable default).
- › `with_structured_output(Route)` — forces the reply into a validated Route object instead of free text.

Alternatives we could have used

- Keyword rules (free, brittle) · embedding similarity (cheap, weak on multi-part) · plan-then-execute (fewer calls, can't adapt).

Phase 3.1 — RAG agent

Putting every document into the prompt doesn't scale. RAG sends the LLM only the few chunks that are relevant.

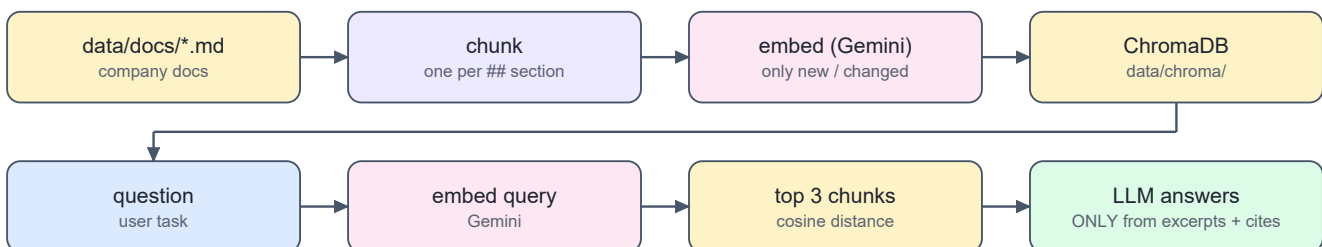


Figure 4 — RAG: ingest once (top row), query every call (bottom row).

Important syntax

```

agents/rag.py

collection.upsert(
    ids=[ids[i] for i in changed], # insert or update by id -> safe to re-run
    documents=[docs[i] for i in changed],
    metadatas=[metas[i] for i in changed])

r = collection.query(query_texts=[query], n_results=k)
[{"source": m["source"], "text": d, "distance": round(dist, 3)}
 for m, d, dist in zip(r["metadatas"][0], r["documents"][0], r["distances"][0])]

```

- › `upsert` — insert-or-update by id, so re-indexing never duplicates; only changed chunks are embedded (API cost).
- › `query_texts=[...]` — Chroma calls our `GeminiEmbeddings.embed_query`, so it can never fall back to a local model.
- › `zip(...)[0]` — results are lists per query; `[0]` takes the first (only) query, zip pairs source, text and distance.

Alternatives we could have used

- Size-based splitter (`RecursiveCharacterTextSplitter`) for unstructured text · agentic RAG (search as a tool, more flexible, slower) · distance threshold to skip the LLM when nothing matches.

Phase 3.2 — Tool calling (Document agent)

Agents now **act**: the LLM asks for a tool, our code runs it, the result goes back, repeat until a final answer. The same loop powers all six tool agents.

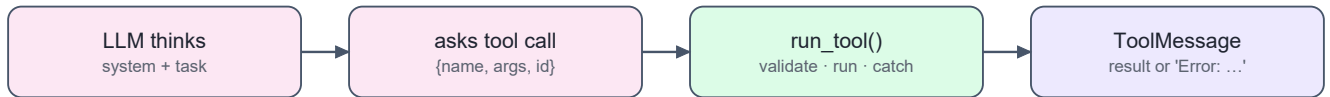


Figure 5 — The agent loop. It repeats (max 8 turns) until the LLM replies without a tool call.

Important syntax

agents/document.py · agents/tool_agent.py

```
@tool
def read_document(filename: str) -> str:
    """Extract text from a .docx, .pdf, .txt or .md file in the workspace."""
    ...

def _safe_path(filename: str) -> Path:
    path = (workspace / filename).resolve()
    if path.parent != workspace:
        # blocks ../secrets.txt, C:/Windows/...
        raise ValueError("must be a plain file name inside the workspace")
    return path

for _ in range(max_turns):
    ai = llm.invoke(messages)
    messages.append(ai)
    if not ai.tool_calls:
        return ai.text
    # no tool requested = final answer
    for call in ai.tool_calls:
        messages.append(ToolMessage(run_tool(name, by_name, call),
                                     tool_call_id=call["id"]))
```

- > **@tool** — turns the function name, type hints and docstring into a JSON schema — **the docstring is a prompt**.
- > **.resolve() / path.parent** — normalises ../ and absolute paths, then insists the file sits directly in the workspace.
- > **ai.tool_calls** — the list of tools the LLM requested; empty means it is done.
- > **ToolMessage(..., tool_call_id=...)** — links each result to the exact call that asked for it.
- > **for _ in range(max_turns)** — a hard cap so a confused LLM can't loop forever.

Alternatives we could have used

- LangGraph's prebuilt `create_react_agent` (same loop, prebuilt) · each agent as its own subgraph.

Phase 3.3 / 3.4 — Excel and PPT agents

LLMs are unreliable at arithmetic and precise layout. So the LLM decides **what** to do, and Python does the **exact** work: totals in Excel, slide layout and limits in PowerPoint.

Important syntax

agents/excel.py · agents/ppt.py

```
@tool
def analyze_sheet(filename: str, value_column: str, group_by: str = "",
                  sheet_name: str = "") -> str:
    """Compute sum, average, min, max and count of a numeric column,
    optionally per group. ALWAYS use this instead of doing maths yourself."""

class Slide(BaseModel):
    title: str
    bullets: list[str] = Field(default_factory=list,
                               description=f"At most {MAX_BULLETS} short bullet points.")
    notes: str = Field(default="", description="Optional speaker notes.")
```

- › `group_by: str = ""` — a default value makes the argument optional in the schema the LLM sees.
- › `"ALWAYS use this..."` — rules inside the docstring steer the LLM at the moment it chooses a tool.
- › `Slide(BaseModel)` — a structured argument: the LLM sends {title, bullets, notes}; bad shapes are rejected before our code runs.

Why we used it

- Prompt limits make good behaviour likely; tool checks make bad output impossible — use both.
- openpyxl does not calculate formulas, so `analyze_sheet` computes in Python and skips formula cells.

Phase 3.5 — Browser agent (untrusted input)

The first agent that reads the open internet. Two read-only tools: `web_search` (Tavily) and `fetch_page` (httpx).

Important syntax

agents/browser.py

```
def _check_url(url: str) -> None:
    parsed = urlparse(url)
    if parsed.scheme not in ("http", "https") or not parsed.hostname:
        raise ValueError(f"Only http(s) URLs are allowed, got: {url}")
    for _, sockaddr in socket.getaddrinfo(parsed.hostname, None):
        ip = ipaddress.ip_address(sockaddr[0])
        if not ip.is_global:
            # loopback, LAN, cloud metadata ...
            raise ValueError(f"Blocked non-public address {ip}")
```

- › `socket.getaddrinfo` — resolves the hostname to every IP it points at, so a name can't hide a private address.
- › `for *, sockaddr in ...` — star-unpacking: ignore the first four tuple fields, keep the last one.
- › `ip.is_global` — true only for public internet addresses — blocks SSRF to localhost, the LAN and 169.254.169.254.
- › `follow_redirects=False` — (in `_download`) every redirect hop is re-checked by hand.

Why we used it

- Least privilege: even a fooled agent can only read, never email or delete.
- Page text is wrapped as UNTRUSTED data, and the prompt says never follow instructions inside it.

Phase 3.6 / 3.7 — Email and Calendar agents

These act toward other people, so they are split into steps. The agents can only **draft** and **propose**; `send_draft()` and `apply_change()` are plain functions that only the human approval path calls. Calendar time maths (free slots, overlaps) is done in Python, and today's date is injected into the prompt on every call.

Why we used it

- A phishing email told the AI to forward the inbox; it refused and warned the user. Even if fooled, the worst outcome is a draft you still must approve.
- Test the **state** (drafts, pending list), not the agent's words — an agent once claimed a cancellation it never made. A shared HONESTY_RULE now covers every agent.

Alternatives we could have used

- Gmail API / Google Calendar API (need OAuth) · IMAP/SMTP (app password) · Microsoft Graph · CalDAV.

Phase 4 — Agent-to-agent workflows

Real tasks are chains (read a report → make slides) and some parts are independent. Each turn the router returns every step that can run **now**; steps in the same batch run in parallel.

Important syntax

```
orchestrator.py

def route_next(state: State):
    if not state.get("plan"):
        return "approval"
    return [Send(p["agent"], {"task": p["task"]}) for p in state["plan"]]

builder.add_conditional_edges("orchestrator", route_next, [*AGENTS, "approval"])

def merge_dict(old: dict, new):
    return {} if new == RESET else {**(old or {}), **new}

agent_results: Annotated[dict, merge_dict]  # parallel writes are merged
```

- › **Send(node, payload)** — starts a node with only that payload; a list of Sends runs them in the same step (parallel).
- › **add_conditional_edges(src, fn, targets)** — fn looks at the State and returns where to go next.
- › **{**old, **new}** — dict unpacking: merge two dicts; later keys win.
- › **Annotated[dict, merge_dict]** — without a reducer, two agents writing at once raise an error; with it they merge.

Why we used it

- The router lists **inputs** and Python copies the earlier results word for word, so facts are never re-typed or shortened by the LLM.
- Plumbing belongs in code: if the router forgets inputs, Python attaches all earlier results.

Alternatives we could have used

- Router rewrites facts into the task (lossy) · every agent gets everything (noisy) · plan-then-execute (cheaper, can't adapt).

Phase 5 — Tools as a system

27 tools reviewed together. The loop guarantees: errors go back as text, unknown tools are reported, arguments are validated by Pydantic before the function runs, huge output is cut to 20k chars, invented argument names are removed (a bad call from one provider once broke the next provider).

Principle	What it means
The docstring is the prompt	Say when to use the tool and what the arguments look like.
Never trust arguments	Check paths, URLs, addresses and limits inside the tool itself.
Short string results	Including failures (Error: Type: message), so the LLM can recover.
Least privilege	Dangerous actions are proposals or human-only functions.
Tools do exact work	Maths, time, files in Python; the LLM only decides what to do.

Principle	What it means
Consistency	Partial updates everywhere, so the LLM can guess how a new tool behaves.

Phase 6 — FastAPI backend

A thin HTTP layer: validate, authenticate, translate errors. All intelligence stays in the graph.

Important syntax

```
api.py (simplified)

@app.post("/tasks/stream")
def stream_task(req: TaskRequest, user: str = Depends(current_user)):
    ...

def events():
    # a plain generator
    yield _sse("thread", {"thread_id": thread_id})
    for update in traced_stream(graph_input, config, user):
        for node, out in update.items():
            yield _sse("agent", {"agent": node, "result": ...})

return StreamingResponse(events(), media_type="text/event-stream")
```

- › **def**, not **async def** — agents make blocking LLM calls; FastAPI runs **def** endpoints in a thread pool so the server stays responsive.
- › **Depends(current_user)** — dependency injection: FastAPI runs the auth check before the endpoint and passes the user in.
- › **req: TaskRequest** — Pydantic validates the body (1–4,000 chars) — bad input gets 422 before any LLM call.
- › **yield** — each yield sends one SSE event (event: X / data: {json}) as soon as a node finishes.

Alternatives we could have used

- WebSockets (two-way, more complex) · sse-starlette package · async agents with **ainvoke** (more efficient at high load, bigger rewrite).

Phase 7 — React frontend (EDITH)

The orchestrator is drawn as a hub with the agents in a ring; dispatched agents glow, finished ones turn green, results stream into the Mission Log.

Important syntax

```
frontend/src/api.js

const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
let buffer = "";
for (;;) {
    const { value, done } = await reader.read();
    if (done) return;
    buffer += value;
    let end;
    while ((end = buffer.indexOf("\n\n")) >= 0) { // blank line ends an event
        const block = buffer.slice(0, end);
        buffer = buffer.slice(end + 2);
        ... parse "event:" and "data:" lines ...
        if (data) onEvent(event, JSON.parse(data));
    }
}
```

- › **fetch + getReader()** — the browser's EventSource only supports GET; our stream is a POST, so we read the body ourselves.
- › **TextDecoderStream** — turns the raw bytes into text as they arrive.
- › **const { value, done } = ...** — object destructuring of the read result.

> `buffer / indexOf("\n\n")` — network chunks can split an event, so we keep a buffer and cut on blank lines.

Why we used it

- Answers are untrusted (they may quote web pages or emails), so they are never inserted as HTML.
- A Stop button uses AbortController to cancel the fetch.

Phase 8 — Memory, state and persistence

Every request used to start from zero. Now: short-term memory per thread (checkpointer), long-term memory about you (profile), and a history of past missions.

Important syntax

orchestrator.py · memory.py

```
graph = builder.compile(checkpointer=SqliteSaver(conn))
config = {"configurable": {"thread_id": thread_id}}

def new_request(message):
    return {"messages": [HumanMessage(message)], # appended to the conversation
            "agent_results": RESET, "history": RESET, "turns": 0, ...}
```

- > `checkpointer=SqliteSaver(conn)` — saves the full State after every node into data/memory.sqlite.
- > `{"configurable": {"thread_id": ...}}` — the key for a conversation; invoking again with it continues from the saved state.
- > `RESET` — merging reducers can't be cleared with `{}` or `[]`, so custom reducers understand a `RESET` value for per-run fields.

Alternatives we could have used

- MemorySaver (RAM only) · PostgresSaver (production) · automatic memory (LangMem, mem0) — we chose an explicit profile you control, because a wrong or injected memory would persist forever.

Phase 9 — Human approval (human-in-the-loop)

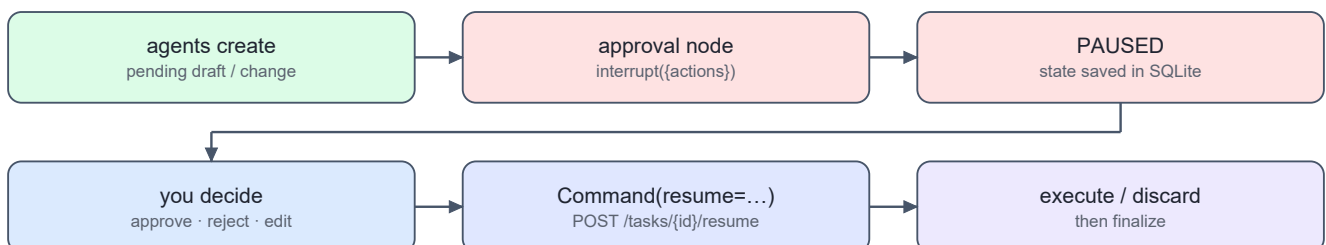


Figure 6 — Approval flow. Anything not explicitly approved is rejected.

Important syntax

orchestrator.py

```
def approval(state):
    ours = [a for a in list_pending() if a["id"] not in state["approval_baseline"]]
    if not ours:
        return {"approvals": []}
    decisions = interrupt({"actions": ours}) # PAUSE here
    for action in ours:
        d = decisions.get(action["id"], "reject") # safe default
        ...execute(action["id"], edits) or discard(action["id"])...

graph.invoke(Command(resume={"email:d1": "approve"}), config)
```

- > `interrupt(value)` — stops the run, saves the State, and sends value to the client; on resume it returns the human's decisions.

- › `Command(resume=...)` — continues the paused thread with the decisions.
- › `decisions.get(id, "reject")` — anything the human didn't approve is rejected.
- › **re-run from the start** — on resume the node runs again, so everything before `interrupt()` must be safe to repeat — that's why it has no LLM.

Why we used it

- An LLM can misunderstand or be manipulated; sending, cancelling and deleting can't be undone.
- Proposals are idempotent: an LLM retrying the same proposal gets the existing id, not a duplicate.

Phase 10 — Reliability

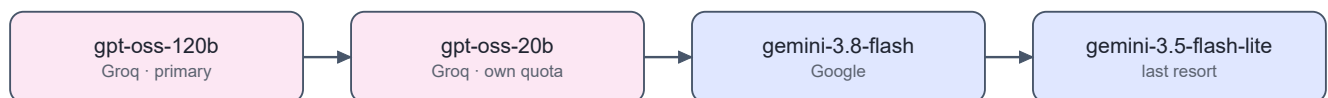


Figure 7 — Fallback order. A failing model is skipped while it cools down (circuit breaker).

Important syntax

llm.py (simplified)

```

def classify(error) -> tuple[str, float]:
    if "per day" in text: return "daily_limit", 1800
    if "429" in text:     return "rate_limit", 20      # or "try again in Xs"
    if "503" in text:     return "unavailable", 15
    if "401" in text:     return "auth", 600
    return "bad_request", 0

for name, runnable in self.options:
    if name in given_up or cooldown_left(name):
        continue                                # circuit open: skip
    try:
        return runnable.invoke(messages, config, **kwargs)
    except Exception as e:
        kind, seconds = classify(e)
        _cool(name, seconds)
  
```

- › `tuple[str, float]` — the function returns two values: the error kind and how long to cool down.
- › `continue` — skip a model that is cooling down instead of wasting a round trip on it.
- › `**kwargs` — passes any extra keyword arguments straight through to the real model.
- › `max_retries=0 (on each model)` — we own retries; two retry layers would multiply the delays.

Important syntax

orchestrator.py

```

def guarded(name, node):
    def run_agent(state):
        try:
            return node(state)
        except GraphBubbleUp:      # LangGraph control flow (interrupts)
            raise
        except Exception as e:
            return {"agent_results": {name: f"ERROR: {name} failed ({e})"}}
    return run_agent
  
```

- › **def inside def** — a closure: `run_agent` remembers `name` and `node` from the outer function (a decorator-style wrapper).
- › `except GraphBubbleUp: raise` — re-raise LangGraph's own signals; swallowing them would break pausing.

Guard	Stops
MAX_TURNS = 6	orchestrator ↔ agent loops

Guard	Stops
MAX_RUN_SECONDS = 300	a request that runs too long
max_turns = 8 (tool loop)	an agent calling tools forever
each agent once per request	the router re-calling agents
MAX_WAIT = 45 s, MAX_ROUNDS = 3	endless waiting on rate limits
timeout = 30 s, fetch_page 10 s	a hung network call
verify_outputs()	an agent claiming a file it never created

Alternatives we could have used

- LiteLLM Router (cooldowns as a library) · tenacity (retry decorators) · LangGraph RetryPolicy (re-runs a whole node, could repeat tool calls) · LangSmith / Langfuse / OpenTelemetry for tracing.

6. Follow-up features

Built after Phase 10 to make the project multi-user, robust and better-looking.

User accounts

Important syntax

auth.py · userdata.py

```
def _hash_password(password: str, salt: bytes) -> bytes:
    return hashlib.scrypt(password.encode(), salt=salt, **SCRYPT)

token = secrets.token_urlsafe(32)                # given to the browser
db_value = hashlib.sha256(token.encode()).hexdigest() # only this is stored

CURRENT_USER = contextvars.ContextVar("current_user", default=LOCAL)
```

- › **hashlib.scrypt** — slow and memory-hungry on purpose, so a stolen database is expensive to brute-force.
- › **secrets.token_urlsafe** — cryptographically random session token.
- › **sha256 of the token** — a stolen database can't be used to log in.
- › **ContextVar** — “who is asking” travels with the request into deep code (tools, mailbox) without passing a user parameter everywhere.

Why we used it

- Same error for unknown user and wrong password (no username discovery); 5 failures lock for 5 minutes.
- Other users' threads return 404, not 403 — nobody can confirm a thread exists.

Alternatives we could have used

- JWT (stateless, but can't be revoked early) · OAuth “Sign in with Google” (no passwords stored, needs Google Cloud) · bcrypt / argon2.

File locking and atomic writes

Important syntax

jsonstore.py

```
@contextmanager
def locked(path):
    with _guard:
        lock = _locks[str(Path(path).resolve())]
    with lock:
        yield

def write_json(path, data):
    tmp.write_text(json.dumps(data, indent=2), encoding="utf-8")
    os.replace(tmp, path) # atomic: old file or new file, never a mix
```

- › **@contextmanager + yield** — turns a function into a with-block: code before yield runs on enter, after on exit.
- › **threading.RLock** — re-entrant lock — the same thread can nest it safely.
- › **os.replace** — atomic swap; fixes the corrupted mailbox reproduced with 20 concurrent drafts.

Alternatives we could have used

- portalocker (OS file locks, multi-process) · a real database, whose transactions do both jobs.

Safe Markdown in the UI

Important syntax

```
frontend/src/App.jsx
```

```
<Markdown remarkPlugins={[remarkGfm]} skipHtml components={MD_COMPONENTS}>
  {text}
</Markdown>
```

- › `remarkPlugins=[remarkGfm]` — adds GitHub tables, strikethrough and task lists.
- › `skipHtml` — drops raw HTML from LLM output — no `<script>`, no `onerror`.
- › `components={...}` — custom renderers, e.g. links open in a new tab with `rel="noopener noreferrer"`.

Edit a draft before approving

A decision can carry edits: `{"decision": "approve", "edits": {"subject": ..., "body": ...}}`. Only subject and body are editable — never the recipient, refused both by the API model (`Literal["subject", "body"]` → 422) and by `execute()`.

Long documents and long conversations

Important syntax

```
agents/document.py · orchestrator.py
```

```
for i, part in enumerate(parts, 1):                # MAP
    notes.append(summarizer().invoke([SystemMessage("≤8 bullets, keep numbers"),
                                     HumanMessage(f"Part {i} of {len(parts)}: \n\n{part}"))].text)
combined = summarizer().invoke([...join the notes...]).text # REDUCE

# conversation compaction (compact node):
return {"summary": summary, "messages": [RemoveMessage(id=m.id) for m in old]}
```

- › `enumerate(parts, 1)` — loops with a counter that starts at 1.
- › `map-reduce` — summarize each ≤ 10 k-char part, then combine: bounded cost (max 15 parts).
- › `RemoveMessage(id=...)` — understood by `add_messages`: those messages leave the saved state after being folded into the summary.

Alternatives we could have used

- RAG over the document (better for specific questions) · a 1M-token model (simple, costly, rate-limited) · “refine” summaries (sequential, slower) · just trimming old turns (cheap, forgets).

Designed presentations

Decks used to be the plain 4:3 default template with bullets on every slide. Now the LLM still writes **structured content**, but Python draws a **designed** 16:9 slide: a colour theme, accent shapes, big-number cards, native charts and live slide numbers. The LLM chooses a **layout** per slide; it never places shapes.



Figure 8 — A deck made by create_presentation (theme “ocean”), exported to images through PowerPoint: title, stats, bullets, bar chart + takeaways, pie chart, two columns, quote and section slides.

Layout	Use it for	What Python draws
bullets	normal lists	themed square bullets, “Label: detail” in bold, font size by amount of text
two_column	before/after, pros/cons	two soft cards with headings
stats	1–4 key numbers	cards with a big number, a label and an accent strip
chart	data by category or time	native bar / line / pie chart (editable in PowerPoint) + up to 3 takeaways
quote	one key message	large quotation mark, italic serif text
section	dividers in long decks	full-colour slide with a large title

Important syntax

agents/ppt.py (simplified)

```
class Slide(BaseModel):
    title: str
    layout: Literal["bullets", "two_column", "stats", "chart", "quote", "section"] = "bullets"
    bullets: list[str] = Field(default_factory=list)
    stats: list[Stat] = Field(default_factory=list)      # Stat = {value, label}
    chart: Chart | None = None                        # Chart = {type, categories, values}
    ...

data = CategoryChartData()
data.categories = c.categories
data.add_series(c.series_name or "Value", c.values)
frame = slide.shapes.add_chart(XL_CHART_TYPE.COLUMN_CLUSTERED, x, y, w, h, data)
frame.chart.value_axis.minimum_scale = 0             # bars start at zero

fld = etree.SubElement(p._p, qn("a:fld"), type="slidenum", id="{...}") # live slide number
prs.core_properties.category = theme                 # theme saved in the file
```

- › `Literal[...] = "bullets"` — the LLM can only pick a real layout; the default keeps old calls working.
- › `Chart | None = None` — an optional nested model: the tool schema tells the LLM its exact shape.
- › `CategoryChartData / add_chart` — python-pptx builds a real PowerPoint chart (with its own data sheet), not a picture.
- › `etree.SubElement(... "a:fld")` — python-pptx has no slide-number API, so we add the XML element PowerPoint uses; it updates itself after moves and deletes.
- › `core_properties.category` — stores the theme inside the file, so slides added later use the same colours.

Why we used it

- No new dependency: python-pptx already supports shapes, colours, charts and XML fields — the design is our code.
- Native charts stay editable; the tools still enforce limits (≤ 6 bullets, ≤ 4 stats, values match categories).
- Checked visually through PowerPoint, which caught labels like "88." and a bar axis starting at 65.

Alternatives we could have used

- A designer-made .pptx template (best look, needs a designed file) · matplotlib chart images (more chart types, not editable) · Aspose.Slides (paid) · Google Slides API (OAuth) · pptxgenjs (JavaScript) · stock photos per slide (needs an API key).

Uploading documents (workspace + RAG)

The **Add document** button in the request box sends a file to **PUT /files/{name}**. It is saved in your workspace, so every agent can use it, and PDF / DOCX / TXT / MD files are also indexed into the RAG knowledge base **for you only**. The next request starts with "Attached documents: ..." so the agents know the file names.



Figure 9 — Upload flow. RAG search then returns company docs + your own uploads, never another user's.

Important syntax

api.py · agents/rag.py

```
async for part in request.stream():          # count while reading
    data += part
    if len(data) > MAX_UPLOAD:
        raise HTTPException(413, "File is larger than 20 MB.")
if suffix in MAGIC and not data.startswith(MAGIC[suffix]): # %PDF, PK (Office zip)
    raise HTTPException(415, f"This doesn't look like a real {suffix} file.")

collection.query(query_texts=[query], n_results=k,
                 where={"owner": {"$in": ["shared", CURRENT_USER.get()]}})
```

- > `request.stream()` — reads the body in pieces, so a fake Content-Length can't make the server load a huge file.
- > `data.startswith(MAGIC[...])` — checks the file's real signature, not just its name.
- > `where={"owner": {"$in": [...]}}` — Chroma metadata filter: company docs plus the current user's uploads.
- > `run_in_threadpool(...)` — saving and embedding block, so they run off the async event loop.

Why we used it

- One index with an owner field keeps users' documents private without one collection per user.
- Raw-body upload needs no new dependency; only new or changed chunks are embedded (API quota).

Alternatives we could have used

- Multipart form upload (needs python-multipart) · one Chroma collection per user · OCR for scanned PDFs (not done) · deleting an upload and its chunks (not done yet).

Document Editor (templates and surgical edits)

A **separate agent** for changing existing documents, so routing is precise: “rewrite / edit / put this in our template” goes to **document_editor** and nowhere else, while the Document agent reads and creates new files, and RAG only answers questions. Typical use: upload a PDF and your company template or letterhead .docx, then ask to rewrite the PDF into it.

Tool	What it does	Keeps
fill_template	copies the template .docx and writes the new content below what it already contains, in the template's styles	logo, header, footer, fonts, margins, page setup
edit_document	exact find → replace in body, tables (also nested), headers and footers; all-or-nothing	bold, sizes, fonts, styles of the edited text

Important syntax

agents/editor.py (simplified)

```
doc = Document(template)          # a copy of the user's template
for p in reversed(doc.paragraphs): # drop trailing empty lines...
    if p.text.strip() or p._p.xpath("./w:drawing | ./w:pict"):
        break                      # ...but keep text and images (a logo)
    body.remove(p._p)
_write_markdown(doc, content)      # inserted before the section settings
doc.save(free_path(out))           # never overwrite: name_2.docx

for run in paragraph.runs:         # surgical edit: replace inside a run
    if find in run.text:           # -> its formatting is untouched
        run.text = run.text.replace(find, new)
```

- > `p._p.xpath("./w:drawing")` — looks into the paragraph's XML for an image, so a logo placed in the body is never deleted.

- › `_write_markdown(doc, ...)` — uses the template's own Heading / List Bullet styles; a template without them gets bold headings and “•” bullets instead.
- › `run.text = ...` — Word stores text in runs (pieces with one formatting); replacing inside a run keeps its bold, size and font.
- › `all-or-nothing` — if any find text is missing, nothing is saved and the agent is told exactly what wasn't found.

Why we used it

- Precise routing: one agent per kind of job; the router check sends “rewrite onto our letterhead” and “change the date in offer_letter.docx” to document_editor only.
- A PDF can't be edited in place, so its content is re-written into the user's own .docx template.
- Both tools save a new file, so the user's originals are never lost.

Alternatives we could have used

- Editing PDFs directly (PyMuPDF redaction + overlay: fragile layout, licence AGPL) · docxtpl / Jinja placeholders in the template (needs a prepared template) · Word/LibreOffice automation (needs the app) · PDF output via Word or LibreOffice export (not done yet).

7. Security summary

Defence in layers: the prompt makes good behaviour likely; the code makes bad behaviour impossible.

Risk	Where	Defence
Path traversal (../secrets)	file tools, downloads	<code>_safe_path</code> : plain names inside the user's workspace; tested at two layers
Prompt injection	web pages, emails	content wrapped as UNTRUSTED; read-only / draft-only tools; human approval
SSRF (localhost, LAN, metadata IP)	<code>fetch_page</code>	public IPs only; every redirect re-checked; size and time limits
Irreversible actions	send, cancel, delete	agents only propose; <code>interrupt()</code> waits for you; reject by default
Forged decisions	resume API	409 if not paused, 422 for unknown ids, recipient never editable
Password theft	auth	salted script, hashed session tokens, lockout, uniform error messages
Cross-user access	threads, files, approvals	owner column + <code>ContextVar</code> ; 404 for others' threads
Malicious uploads	<code>PUT /files</code>	type allow-list, 20 MB cap while reading, file signature check, safe names, never overwrite
XSS	UI	react-markdown with <code>skipHtml</code> ; never <code>innerHTML</code>
Timing attack on API key	auth	<code>secrets.compare_digest</code>
Data corruption	JSON stores	per-file locks + <code>atomic os.replace</code> + transactions
Leaking internals	errors	503 for provider trouble, 500 for bugs, details only in server log

8. Key lessons from the bugs we hit

Lesson	What taught it
Test the state, not the words	Calendar agent said it proposed a cancellation it never made.
Plumbing belongs in code	The router kept forgetting inputs and re-calling agents despite prompt rules.
In a chain, the reply IS the data	Document → PPT looped 6 turns because the doc agent only said “I created a file”.
Every provider must accept the history	One model's malformed call made Gemini reject the whole conversation.
Pin model versions	“-latest” aliases hid Gemini-3 handling and broke cross-provider tool calls.
Don't lump errors together	An arrow character crashed logging, but showed as “providers unavailable”.
Every action must be safe to request twice	A retrying model proposed the same cancellation twice.
Look at the output, not just the data	Rendering the deck showed “88.” labels and a chart axis starting at 65.
Check the library's defaults	Chroma was silently configured with a local embedding model.
Tests must never touch real data	Early tests reset the real mailbox and polluted history.
Don't name files after stdlib modules	email.py, calendar.py, trace.py all shadow built-ins on import.

9. Running the project

Keys needed (.env)

Variable	Used for
GROQ_API_KEY	main LLMs
GOOGLE_API_KEY	fallback LLMs + RAG embeddings
TAVILY_API_KEY	web search
APP_API_KEY (optional)	shared key for local dev

Commands

```
# backend (http://127.0.0.1:8000/docs)
.venv/Scripts/python -u -m uvicorn api:app --port 8000

# frontend (http://localhost:5173)
npm --prefix frontend run dev

# command-line run of the orchestrator
python orchestrator.py

# see what a run did
python trace_view.py
```

Project files

File	Role
orchestrator.py	the LangGraph graph: State, router, approval, finalize, compact, guards
agents/*.py	the 9 agents; tool_agent.py is the shared tool loop
llm.py	ResilientLLM: fallbacks, cooldowns, backoff
api.py	FastAPI endpoints and SSE streaming
memory.py · auth.py · userdata.py	checkpointer and profile · accounts and sessions · per-user paths
approvals.py · jsonstore.py	pending risky actions · locked, atomic JSON files
tracing.py · trace_view.py	JSON-lines trace and its viewer
previews.py	file previews for the UI
frontend/src/	App.jsx (UI), api.js (API + SSE parser)

About the tests

- The test files were removed from the working tree on request (commit b3aa353).
- They are kept in git history: git checkout b08dfaa -- tests restores them; run with python -m tests..